

**MBA
USP
ESALQ**

INGENIERÍA DE DATOS III

Prof. Dr. Jeronymo Marcondes

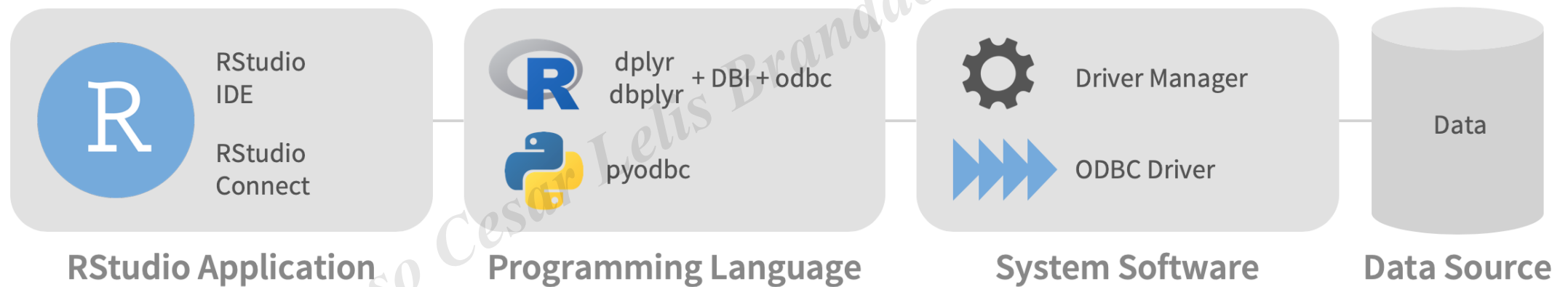
*El profesor es responsable de la idoneidad, originalidad y legalidad de los contenidos didácticos presentados.

Prohibida la reproducción total o parcial, sin autorización. Ley nº 9610/98

ODBC y JDBC

- Enfoque de la clase: ODBC
- Objetivo: utilizar las capacidades del banco de datos para realizar nuestras operaciones.
- Limitaciones de los lenguajes al trabajar con archivos de texto con los datos necesarios.

Ejemplo en R Studio



ODBC y JDBC

- Enfoque de la clase:
 1. Ejecutar las *queries* y llevarlas a *R*.
 2. Utilizar el *dplyr* para realizar las operaciones dentro del *database*.
- Todo eso depende del entendimiento de los paquetes *DBI* y *DPLYR*.

Paquete *DBI*

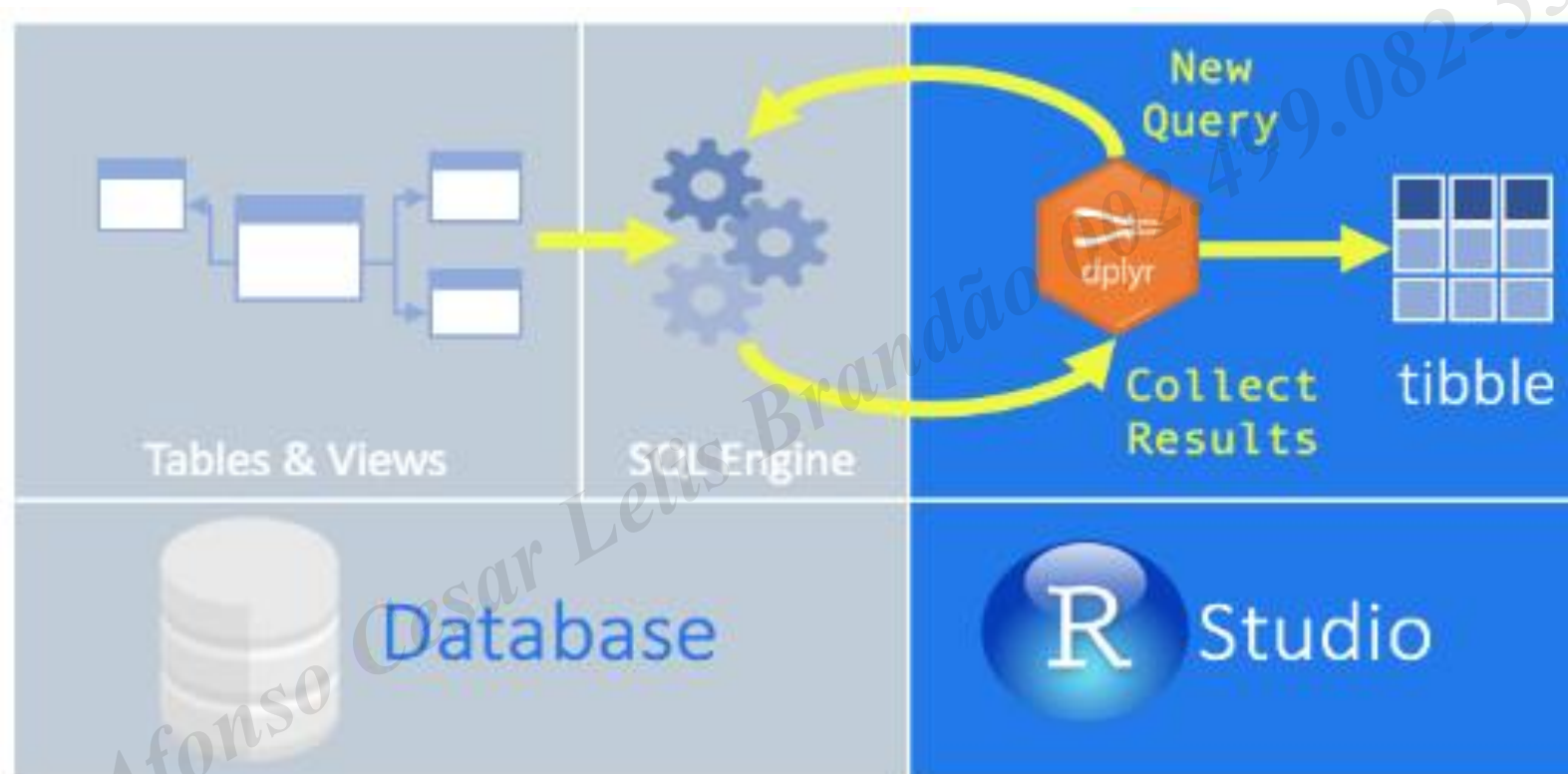
<https://db.rstudio.com/r-packages/dbi/>

DBI

- Permite la división de conexión en un *front-end* y un *back-end*
- Ejecuta *query* en el banco
- Permite que se realice interacción entre otros paquetes y el banco (*dplyr*)

Bases comerciales y *open source*

DBI



Fuente: <https://db.rstudio.com/getting-started/>

DBI - estrutura

```
install.packages("DBI")
```

```
library("DBI")
```

```
con <- DBI::dbConnect(odbc::odbc(),  
                      Driver   = "[your driver's name]",  
                      Server   = "[your server's path]",  
                      UID      = rstudioapi::askForPassword("Database user"),  
                      PWD      = rstudioapi::askForPassword("Database password"),  
                      Port     = 3306)
```

DBI - ejecución de consultas

```
dbListTables(con)
dbWriteTable(con, "mtcars", mtcars)
dbListTables(con)

dbListFields(con, "mtcars")
dbReadTable(con, "mtcars")
```

DBI - ejecución de consultas

```
res <- dbSendQuery(con, "SELECT * FROM mtcars WHERE cyl = 4")  
dbFetch(res)  
dbClearResult(res)
```

DBI - buenas prácticas

- Cerrar la conexión después del uso
- Prestar atención para los *drivers* en entornos corporativos:
<https://db.rstudio.com/best-practices/drivers.>
- Saber lo que está ocurriendo cuando creamos un *dataframe* local con consulta.

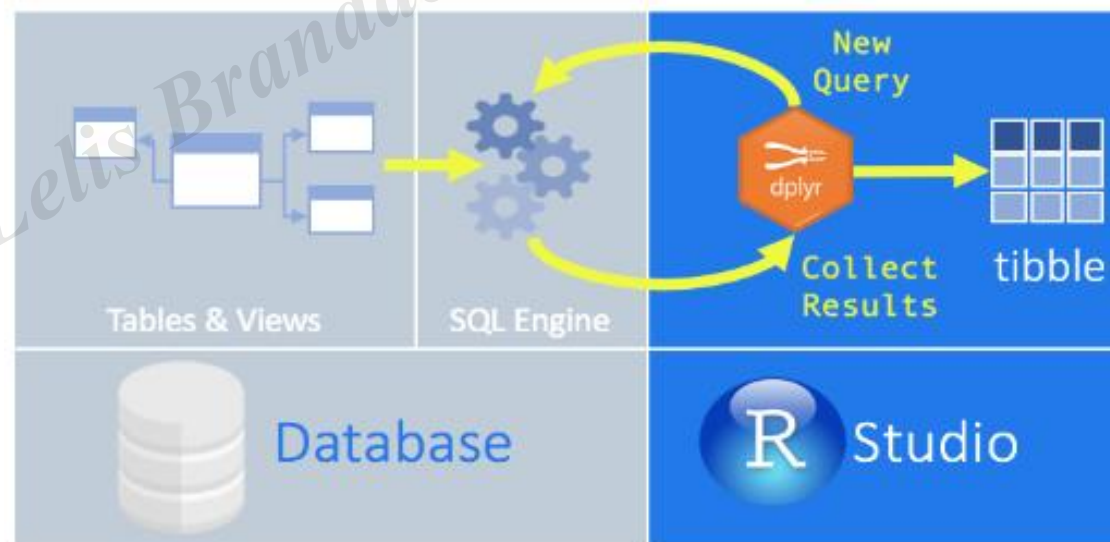
DPLYR

- Principal paquete para *Data Wrangling*.
- Rápido y de fácil manipulación — resuelve más del 90% de los problemas de estructura que pueden surgir.
- Trabaja con *dataframes* que siguen padrones de *tidydata* (*tibble*).

DPLYR

- *Dplyr* permitirá la ejecución de algunas operaciones de *wrangling* directamente en el banco y utilizando el poder computacional del banco sin sobrecargar *R*.

```
install.packages("dplyr")  
library("dplyr")
```



Tibble

- *Hadley Wickham:*

1. Cada variable es una columna
2. Cada observación es una línea
3. Cada unidad observacional es la tabla

DPLYR

- Las principales funciones del *dplyr* son:

1. *Select()*
2. *Arrange()*
3. *Filter()*
4. *Mutate()*
5. *Group_by()*
6. *Summarise()*

DPLYR - estrutura básica

- Uso básico del *select* – selecciona columnas de una tabla

select(dataframe, campos_separados_por_coma)

DPLYR - uso del *pipe*

- Uso del *pipe* – más de una operación del *dplyr* en una única tabla
- Idea de secuencial

dataframe %>% select(campos_separados_por_coma)

DPLYR - exemplo de *select*

```
dim(starwars)
#> [1] 87 14
starwars
#> # A tibble: 87 × 14
#>   name          height  mass hair_color skin_color eye_color birth_year sex
#>   <chr>         <int> <dbl> <chr>      <chr>      <chr>      <dbl> <chr>
#> 1 Luke Skyw...    172    77 blond     fair        blue         19  male
#> 2 C-3PO          167    75 NA        gold        yellow       112  none
#> 3 R2-D2           96    32 NA        white, bl... red          33  none
#> 4 Darth Vad...   202   136 none      white       yellow      41.9  male
#> # ... with 83 more rows, and 6 more variables: gender <chr>,
#> #   homeworld <chr>, species <chr>, films <list>, vehicles <list>,
#> #   starships <list>
```

<https://dplyr.tidyverse.org/articles/dplyr.html>

DPLYR - exemplo de *select*

```
starwars %>% select(hair_color, skin_color, eye_color)
#> # A tibble: 87 × 3
#>   hair_color skin_color eye_color
#>   <chr>      <chr>      <chr>
#> 1 blond      fair        blue
#> 2 NA         gold        yellow
#> 3 NA         white, blue red
#> 4 none       white        yellow
#> # ... with 83 more rows
```

<https://dplyr.tidyverse.org/articles/dplyr.html>

DPLYR - exemplo de *filter*

dataframe %>% filter(campos == “valor determinado”)

dataframe %>% filter(campos > “valor determinado”)

dataframe %>% filter(campos < “valor determinado”)

dataframe %>% filter(campos != “determinado valor”)

DPLYR - exemplo de *filter*

```
filter(starwars, species == "Human")
#> # A tibble: 35 × 14
#>   name      height  mass hair_color skin_color eye_color birth_year sex
#>   <chr>      <int> <dbl> <chr>      <chr>      <chr>      <dbl> <chr>
#> 1 Luke Sky...   172    77 blond      fair       blue        19   male
#> 2 Darth Va...   202   136 none       white      yellow     41.9  male
#> 3 Leia Org...   150    49 brown      light      brown       19   fema...
#> 4 Owen Lars    178   120 brown, gr... light      blue       52   male
#> 5 Beru Whi...   165    75 brown      light      blue       47   fema...
#> 6 Biggs Da...   183    84 black      light      brown       24   male
#> 7 Obi-Wan ...   182    77 auburn, w... fair       blue-gray   57   male
#> 8 Anakin S...   188    84 blond      fair       blue     41.9  male
#> 9 Wilhuff ...   180    NA auburn, g... fair       blue       64   male
#> 10 Han Solo    180    80 brown      fair       brown       29   male
#> # ... with 25 more rows, and 6 more variables: gender <chr>,
#> #   homeworld <chr>, species <chr>, films <list>, vehicles <list>,
#> #   starships <list>
```

<https://dplyr.tidyverse.org/articles/dplyr.html>

DPLYR - ejemplo de *group_by* y *summarise*

dataframe %>% summarise(nueva_variable = función aplicada en variable)

dataframe %>% group_by(variable) %>% summarise(nueva_variable = función aplicada en variable)

DPLYR - exemplo de *group_by* y *summarise*

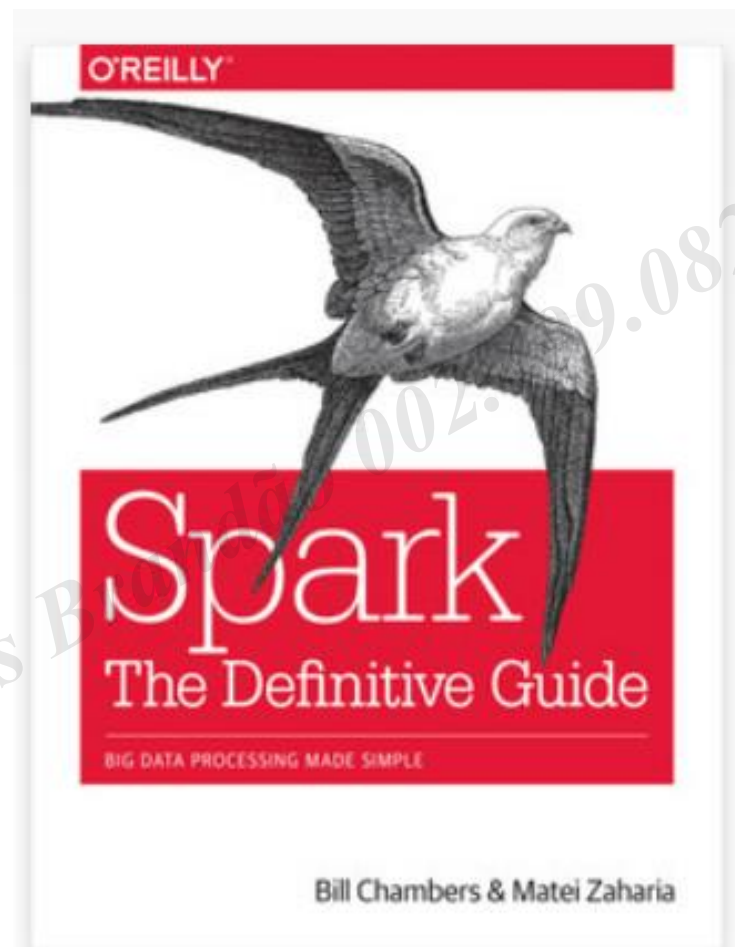
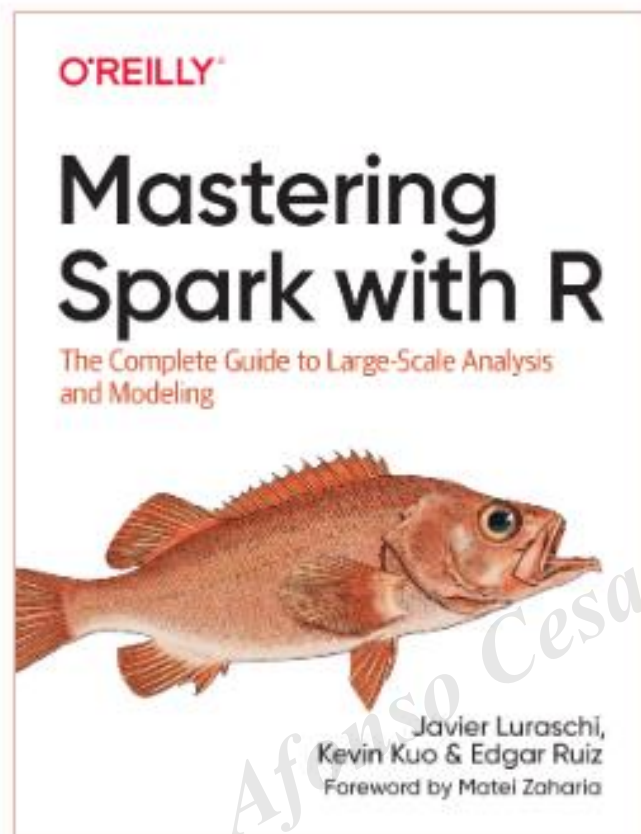
```
starwars %>% summarise(height = mean(height, na.rm = TRUE))  
#> # A tibble: 1 × 1  
#>   height  
#>   <dbl>  
#> 1    174.
```

<https://dplyr.tidyverse.org/articles/dplyr.html>

DPLYR - otros casos

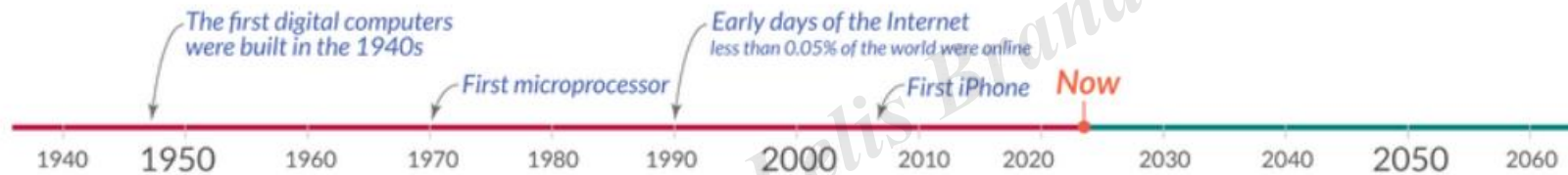
- Amplitud de uso del *dplyr*
- Enfoque de la clase

Apache Spark



Big Data

- Evolución constante de las computadoras



Fuente: <https://ourworldindata.org/brief-history-of-ai>

Big Data

- Computadoras cada vez más potentes = + datos.
- 2005 – velocidad de los datos es demasiado grande.
- Inicie el proceso de computación paralela.

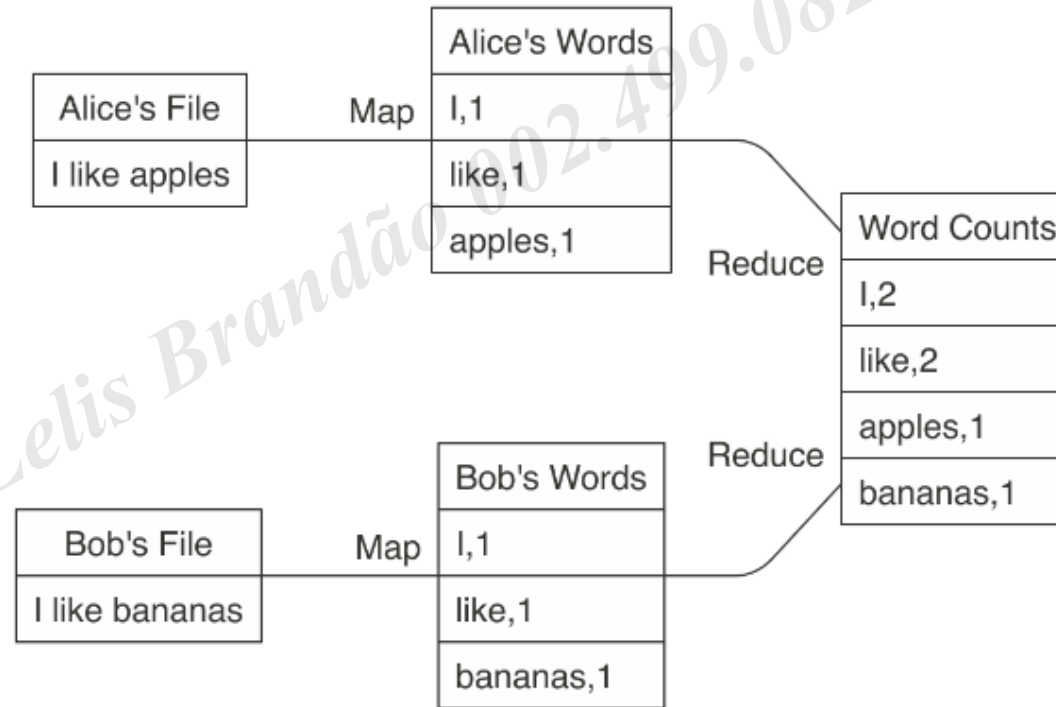
Big Data

- Custo de armazenamento tampoco reduce.
- Sólo paralelar *CPU* no es suficiente.
- Entre a *Google File System*

Hadoop

- *Map Reduce*

- *HDFS*



Apache Spark

- *UC Berkeley*
- *“Spark: Cluster Computing with Working Sets”.*
- *Vantagens y Desvantagens del Map Reduce.*

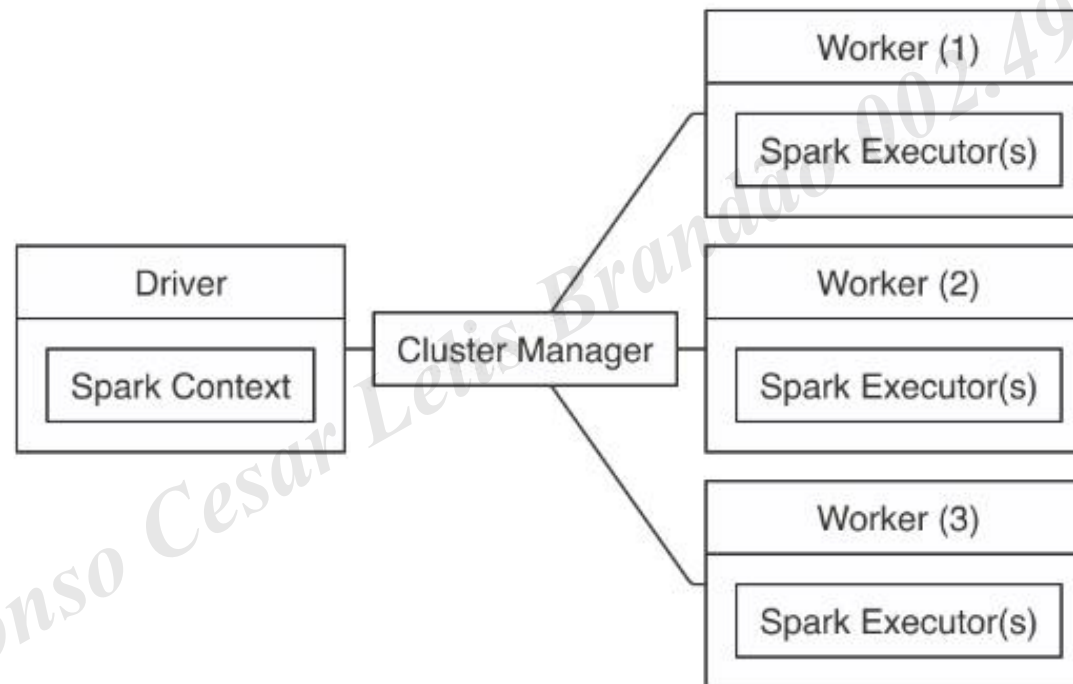
Apache Spark

- Entra *Spark*.
- Posibilidad de procesamiento hecho “*in memory*”.
- Velocidad de ejecución.

Apache Spark

- Es una *Engine* - o sea, sirve para mejorar los más diversos procesos. Se adapta a varios lenguajes.
- Estructura:
 1. *Driver*
 2. *Cluster Manager*
 3. *Executors*

Apache Spark



Apache Spark

- *Cluster Managers: Spark Standalone, YARN, Mesos, and Kubernetes.*
- Nuestro enfoque será en ejecución local.
- Lenguajes soportados: *Scala, Python, R, Java, etc.*

Apache Spark

- Puerta de entrada: *Spark Session*.
- No necesitamos preocuparnos por eso – interfaz con el *R*.
- Concepto del *sparklyr*

Apache Spark con sparklyr

```
install.packages("sparklyr")
```

```
spark_available_versions()
```

```
spark_install(version = "1.6")
```

Apache Spark

- Importante destacar de nuestro ejercicio:

1. Vamos a ejecutar localmente.

2. Más datos = ¡mejor!

Apache Spark

[colab](#)

MBA
USP
ESALQ

Prof. Dr. Jeronymo Marcondes



<https://www.linkedin.com/in/jeronymo-marcondes-585a26186>